

WEEK 5

SESSION 1 : MAXIMUM SUBARRAY

Maximum Subarray (Brute → Optimized)

Task 1: <https://leetcode.com/problems/maximum-subarray/>

Given an integer array `nums`, find the subarray with the largest sum, and return *its sum*.

Example 1:

Input: `nums = [-2,1,-3,4,-1,2,1,-5,4]`

Output: 6

Explanation: The subarray `[4,-1,2,1]` has the largest sum 6.

Example 2:

Input: `nums = [1]`

Output: 1

Explanation: The subarray `[1]` has the largest sum 1.

Example 3:

Input: `nums = [5,4,-1,7,8]`

Output: 23

Explanation: The subarray `[5,4,-1,7,8]` has the largest sum 23.

PROGRAM

```
class Solution {
    public int maxSubArray(int[] nums) {

        int maxSum = nums[0];

        for (int i = 0; i < nums.length; i++) {

            int sum = 0;

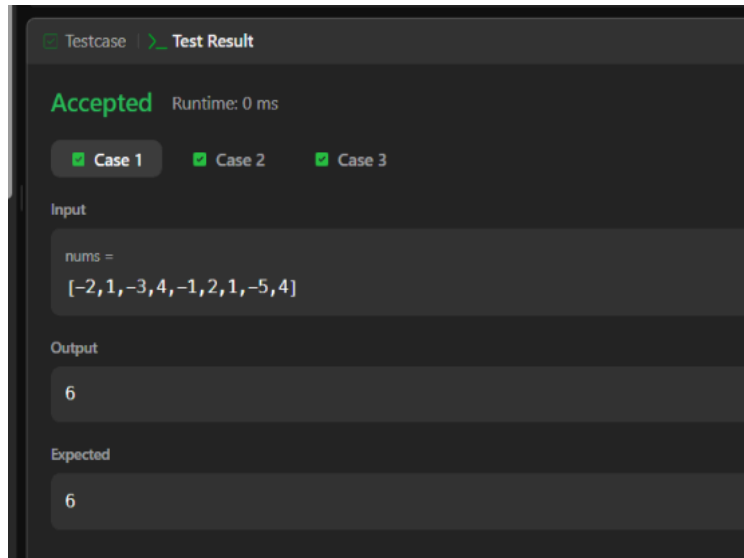
            for (int j = i; j < nums.length; j++) {

                sum += nums[j];

                maxSum = Math.max(maxSum, sum);
            }
        }

        return maxSum;
    }
}
```

OUTPUT



Task 2 : <https://www.hackerrank.com/challenges/the-birthday-bar/problem>

Two children, Lily and Ron, want to share a chocolate bar. Each of the squares has an integer on it.

Lily decides to share a contiguous segment of the bar selected such that:

- The length of the segment matches Ron's birth month, and,
- The sum of the integers on the squares is equal to his birth day.

Determine how many ways she can divide the chocolate.

Example

svg
svg
svg

Lily wants to find segments summing to Ron's birth day, svg with a length equalling his birth month, svg. In this case, there are two segments meeting her criteria: svg and svg.

Function Description

Complete the *birthday* function in the editor below.

birthday has the following parameter(s):

- *int s[n]*: the numbers on each of the squares of chocolate
- *int d*: Ron's birth day
- *int m*: Ron's birth month

Returns

- *int*: the number of ways the bar can be divided

Input Format

The first line contains an integer svg , the number of squares in the chocolate bar.

The second line contains svg space-separated integers svg , the numbers on the chocolate squares where svg .

The third line contains two space-separated integers, svg and svg , Ron's birth day and his birth month.

Constraints

- svg
- svg , where (svg)
- svg
- svg

Sample Input 0

```
5
1 2 1 3 2
3 2
```

Sample Output 0

```
2
```

Explanation 0

Lily wants to give Ron svg squares summing to svg . The following two segments meet the criteria:

[image](#)

Sample Input 1

```
6
1 1 1 1 1 1
3 2
```

Sample Output 1

```
0
```

Explanation 1

Lily only wants to give Ron svg consecutive squares of chocolate whose integers sum to svg . There are no possible pieces satisfying these constraints:

[image](#)

Thus, we print svg as our answer.

Sample Input 2

```
1
4
4 1
```

Sample Output 2

```
1
```

Explanation 2

Lily only wants to give Ron svg square of chocolate with an integer value of svg. Because the only square of chocolate in the bar satisfies this constraint, we print svg as our answer.

```
import java.io.*;
import java.math.*;
import java.security.*;
import java.text.*;
import java.util.*;
import java.util.concurrent.*;
import java.util.function.*;
import java.util.regex.*;
import java.util.stream.*;
import static java.util.stream.Collectors.joining;
import static java.util.stream.Collectors.toList;
```

```
class Result {

    public static int birthday(List<Integer> s, int d, int m) {

        int count = 0;

        for (int i = 0; i <= s.size() - m; i++) {

            int sum = 0;

            for (int j = i; j < i + m; j++) {
                sum += s.get(j);
            }

            if (sum == d) {
                count++;
            }
        }

        return count;
    }
}
```

```

public class Solution {
    public static void main(String[] args) throws IOException {
        BufferedReader bufferedReader = new BufferedReader(new InputStreamReader(System.in));
        BufferedWriter bufferedWriter = new BufferedWriter(new
        FileWriter(System.getenv("OUTPUT_PATH")));

        int n = Integer.parseInt(bufferedReader.readLine().trim());

        List<Integer> s = Stream.of(bufferedReader.readLine().replaceAll("\\s+$", "").split(" "))
            .map(Integer::parseInt)
            .collect(toList());

        String[] firstMultipleInput = bufferedReader.readLine().replaceAll("\\s+$", "").split(" ");

        int d = Integer.parseInt(firstMultipleInput[0]);

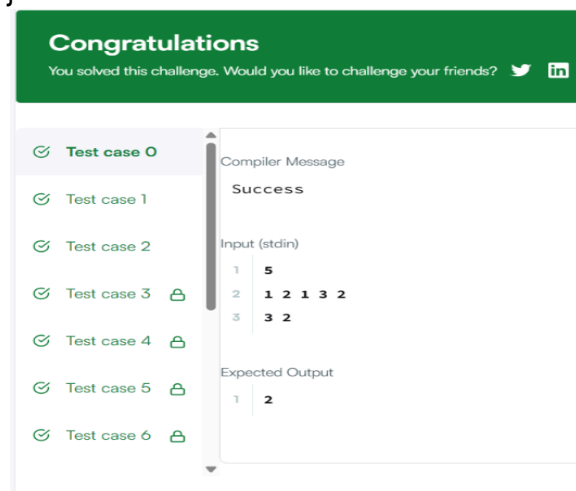
        int m = Integer.parseInt(firstMultipleInput[1]);

        int result = Result.birthday(s, d, m);

        bufferedWriter.write(String.valueOf(result));
        bufferedWriter.newLine();

        bufferedReader.close();
        bufferedWriter.close();
    }
}

```



Kadane's Algorithm

Task 3: <https://www.hackerrank.com/challenges/maxsubarray/>

We define *subsequence* as any subset of an array. We define a *subarray* as a *contiguous subsequence* in an array.

Given an array, find the maximum possible sum among:

1. all nonempty subarrays.
2. all nonempty subsequences.

Print the two values as space-separated integers on one line.

Note that empty subarrays/subsequences should not be considered.

Example

The maximum subarray sum is comprised of elements at indices . Their sum is . The maximum subsequence sum is comprised of elements at indices and their sum is .

Function Description

Complete the *maxSubarray* function in the editor below.

maxSubarray has the following parameter(s):

- *int arr[n]*: an array of integers

Returns

- *int[2]*: the maximum subarray and subsequence sums

Input Format

The first line of input contains a single integer , the number of test cases.

The first line of each test case contains a single integer .

The second line contains space-separated integers where .

The subarray and subsequences you consider should have at least one element.

Sample Input 0

2

4

1 2 3 4

6

2 -1 2 3 4 -5

Sample Output 0

10 10

10 11

Explanation 0

In the first case: The maximum sum for both types of subsequences is just the sum of all the elements since they are all positive.

In the second case: The subarray is the subarray with the maximum sum, and is the subsequence with the maximum sum.

Sample Input 1

1

5

-2 -3 -1 -4 -6

Sample Output 1

-1 -1

Explanation 1

Since all of the numbers are negative, both the maximum subarray and maximum subsequence sums are made up of one element, -1 .

PROGRAM

```
import java.io.;
import java.math.;
import java.security.;
import java.text.;
import java.util.;
import java.util.concurrent.;
import java.util.function.;
import java.util.regex.;
import java.util.stream.*;
import static java.util.stream.Collectors.joining;
import static java.util.stream.Collectors.toList;

class Result {

    public static List<Integer> maxSubarray(List<Integer> arr) {
        // Maximum subarray sum - Kadane's Algorithm
        int currentSum = arr.get(0);
        int maxSubarray = arr.get(0);
        for (int i = 1; i < arr.size(); i++) {
            currentSum = Math.max(arr.get(i), currentSum + arr.get(i));
            maxSubarray = Math.max(maxSubarray, currentSum);
        }
        // Maximum subsequence sum
        int maxSubsequence = 0;
        int maxElement = arr.get(0);
        for (int num : arr) {
            if (num > 0) {
                maxSubsequence += num;
            }
        }
    }
}
```

```

    }
    maxElement = Math.max(maxElement, num);
}
// If all elements are negative, choose the largest element
if (maxSubsequence == 0) {
    maxSubsequence = maxElement;
}
List<Integer> result = new ArrayList<>();
result.add(maxSubarray);
result.add(maxSubsequence);
return result;
}}

public class Solution {
    public static void main(String[] args) throws IOException {
        BufferedReader bufferedReader = new BufferedReader(new InputStreamReader(System.in));
        BufferedWriter bufferedWriter = new BufferedWriter(new
        FileWriter(System.getenv("OUTPUT_PATH")));

        int t = Integer.parseInt(bufferedReader.readLine().trim());

        IntStream.range(0, t).forEach(tltr -> {
            try {
                int n = Integer.parseInt(bufferedReader.readLine().trim());

                List<Integer> arr = Stream.of(bufferedReader.readLine().replaceAll("\\s+$", "").split(" "))
                    .map(Integer::parseInt)
                    .collect(toList());

                List<Integer> result = Result.maxSubarray(arr);

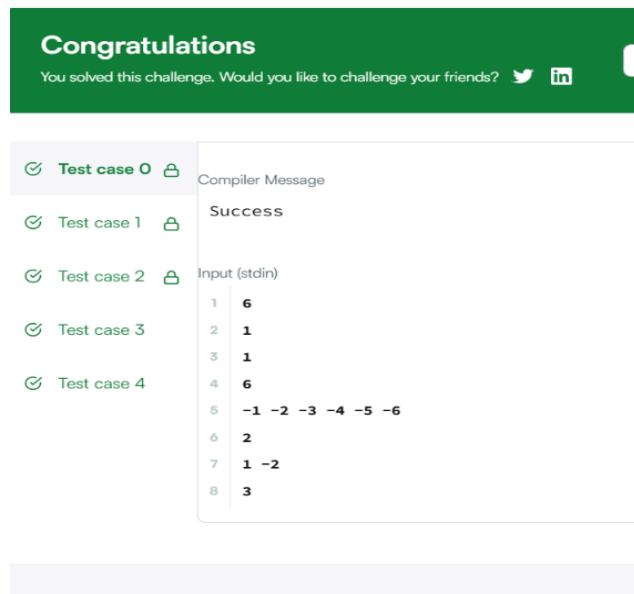
                bufferedWriter.write(
                    result.stream()
                        .map(Object::toString)
                        .collect(joining(" "))
                        + "\n"
                );
            } catch (IOException ex) {
                throw new RuntimeException(ex);
            }
        });
    }
}

```

```

    }
});    bufferedReader.close();
bufferedWriter.close();
}}

```



Task 4: <https://leetcode.com/problems/maximum-sum-circular-subarray/>

Given a **circular integer array** `nums` of length `n`, return *the maximum possible sum of a non-empty subarray of `nums`.*

A **circular array** means the end of the array connects to the beginning of the array. Formally, the next element of `nums[i]` is `nums[(i + 1) % n]` and the previous element of `nums[i]` is `nums[(i - 1 + n) % n]`.

A **subarray** may only include each element of the fixed buffer `nums` at most once. Formally, for a subarray `nums[i], nums[i + 1], ..., nums[j]`, there does not exist `i <= k1, k2 <= j` with `k1 % n == k2 % n`.

Example 1:

Input: `nums = [1,-2,3,-2]`

Output: 3

Explanation: Subarray [3] has maximum sum 3.

Example 2:

Input: `nums = [5,-3,5]`

Output: 10

Explanation: Subarray [5,5] has maximum sum $5 + 5 = 10$.

Example 3:

Input: nums = [-3,-2,-3]

Output: -2

Explanation: Subarray [-2] has maximum sum -2.

Program

```
class Solution {  
    public int maxSubarraySumCircular(int[] nums) {  
        int total = 0;  
        int currentMax = nums[0];  
        int maxSum = nums[0];  
        int currentMin = nums[0];  
        int minSum = nums[0];  
        for (int i = 0; i < nums.length; i++) {  
            total += nums[i];  
            // Kadane's algorithm for maximum subarray  
            if (i > 0) {  
                currentMax = Math.max(nums[i], currentMax + nums[i]);  
                maxSum = Math.max(maxSum, currentMax);  
                // Kadane's algorithm for minimum subarray  
                currentMin = Math.min(nums[i], currentMin + nums[i]);  
                minSum = Math.min(minSum, currentMin);  
            }  
        }  
        // If all elements are negative
```

```

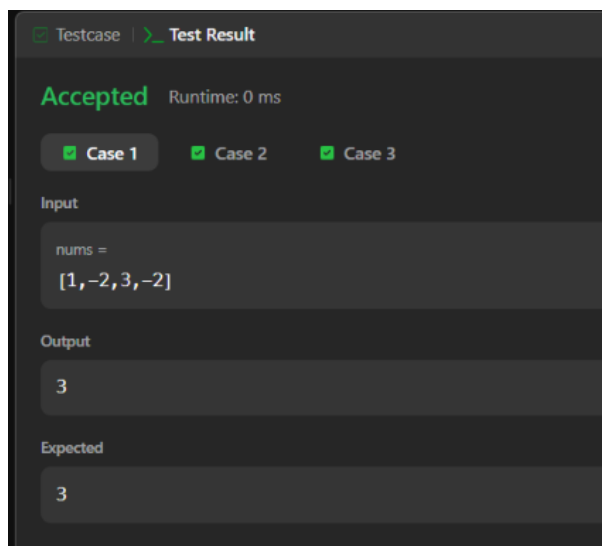
    if (maxSum < 0) {
        return maxSum;
    }

    // Maximum circular sum

    int circularSum = total - minSum;

    return Math.max(maxSum, circularSum);
}
}

```



String Handling Basics

Task 5: <https://leetcode.com/problems/string-to-integer-atoi/>

Implement the `myAtoi(string s)` function, which converts a string to a 32-bit signed integer.

The algorithm for `myAtoi(string s)` is as follows:

1. **Whitespace:** Ignore any leading whitespace (" ").
2. **Signedness:** Determine the sign by checking if the next character is '-' or '+', assuming positivity if neither present.
3. **Conversion:** Read the integer by skipping leading zeros until a non-digit character is encountered or the end of the string is reached. If no digits were read, then the result is 0.
4. **Rounding:** If the integer is out of the 32-bit signed integer range $[-2^{31}, 2^{31} - 1]$, then round the integer to remain in the range. Specifically, integers less than -2^{31} should be rounded to -2^{31} , and integers greater than $2^{31} - 1$ should be rounded to $2^{31} - 1$.

231 - 1.

Return the integer as the final result.

Example 1:

Input: s = "42"

Output: 42

Explanation:

The underlined characters are what is read in and the caret is the current reader position.

Step 1: "42" (no characters read because there is no leading whitespace)

^

Step 2: "42" (no characters read because there is neither a '-' nor '+')

^

Step 3: "42" ("42" is read in)

^

Example 2:

Input: s = "-042"

Output: -42

Explanation:

Step 1: " -042" (leading whitespace is read and ignored)

^

Step 2: " -042" ('-' is read, so the result should be negative)

^

Step 3: " -042" ("042" is read in, leading zeros ignored in the result)

^

Example 3:

Input: s = "1337c0d3"

Output: 1337

Explanation:

Step 1: "1337c0d3" (no characters read because there is no leading whitespace)

^

Step 2: "1337c0d3" (no characters read because there is neither a '-' nor '+')

^

Step 3: "1337c0d3" ("1337" is read in; reading stops because the next character is a non-digit)

^

Example 4:

Input: s = "0-1"

Output: 0

Explanation:

Step 1: "0-1" (no characters read because there is no leading whitespace)

^

Step 2: "0-1" (no characters read because there is neither a '-' nor '+')

^

Step 3: "0-1" ("0" is read in; reading stops because the next character is a non-digit)

^

Example 5:

Input: s = "words and 987"

Output: 0

Explanation:

Reading stops at the first non-digit character 'w'.

Program

```
class Solution {  
    public int myAtoi(String s) {  
        int i = 0;
```

```

int n = s.length();

// Remove leading spaces
while (i < n && s.charAt(i) == ' ') {
    i++;
}

// Check sign
int sign = 1;

if (i < n && (s.charAt(i) == '+' || s.charAt(i) == '-')) {
    if (s.charAt(i) == '-') {
        sign = -1;
    }
    i++;
}

// Convert digits
int result = 0;

while (i < n && Character.isDigit(s.charAt(i))) {
    int digit = s.charAt(i) - '0';

    // Check integer overflow
    if (result > (Integer.MAX_VALUE - digit) / 10) {
        if (sign == 1) {
            return Integer.MAX_VALUE;
        } else {
            return Integer.MIN_VALUE;
        }
    }
    result = result * 10 + digit;
    i++;
}

```

```

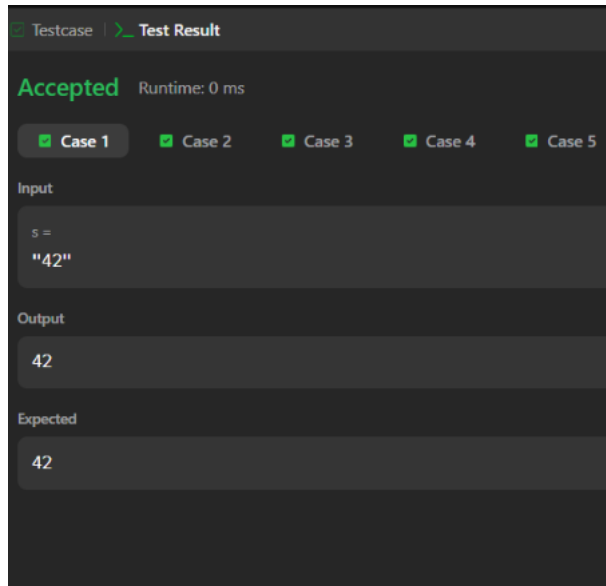
    }

    return result * sign;

}

}

```



Task 6: <https://www.hackerrank.com/challenges/alternating-characters/>

You are given a string containing characters `a` and `b` only. Your task is to change it into a string such that there are no matching adjacent characters. To do this, you are allowed to delete zero or more characters in the string.

Your task is to find the minimum number of required deletions.

Example

Remove an `a` at positions `0` and `2` to make `bab` in `3` deletions.

Function Description

Complete the `alternatingCharacters` function in the editor below.

`alternatingCharacters` has the following parameter(s):

- `string s`: a string

Returns

- `int`: the minimum number of deletions required

Input Format

The first line contains an integer `q`, the number of queries.
The next `q` lines each contain a string `s` to analyze.

Constraints

Each string will consist only of characters `A` and `B`.

Sample Input

5

AAAA

BBBBB

ABABABAB

BABABA

AAABBB

Sample Output

3

4

0

0

4

Explanation

The characters marked red are the ones that can be deleted so that the string does not have matching adjacent characters.

AAAA -> A (3 deletions)

BBBBB -> B (4 deletions)

ABABABAB -> ABABABAB (0 deletions)

BABABA -> BABABA (0 deletions)

AAABBB -> AB (4 deletions)

```
import java.io.;
import java.math.;
import java.security.;
import java.text.;
import java.util.;
import java.util.concurrent.;
```

```

import java.util.function.;
import java.util.regex.;
import java.util.stream.*;
import static java.util.stream.Collectors.joining;
import static java.util.stream.Collectors.toList;

class Result {

    public static int alternatingCharacters(String s) {

        int count = 0;

        for (int i = 1; i < s.length(); i++) {

            if (s.charAt(i) == s.charAt(i - 1)) {

                count++;

            }

        }

        return count;

    }

}

public class Solution {

    public static void main(String[] args) throws IOException {

        BufferedReader bufferedReader = new BufferedReader(new InputStreamReader(System.in));
        BufferedWriter bufferedWriter = new BufferedWriter(new
        FileWriter(System.getenv("OUTPUT_PATH")));

        int q = Integer.parseInt(bufferedReader.readLine().trim());

        IntStream.range(0, q).forEach(qltr -> {

            try {

                String s = bufferedReader.readLine();

                int result = Result.alternatingCharacters(s);    bufferedWriter.write(String.valueOf(result));

                bufferedWriter.newLine();

            } catch (IOException ex) {

                throw new RuntimeException(ex);

            }

        });

        bufferedReader.close();

        bufferedWriter.close();

    }

}

```

}

The screenshot shows a coding platform interface with a list of test cases on the left and a detailed view of the first test case on the right. The test cases are all marked as 'Success'. The first test case (Test case 0) shows the following input and output:

Input (stdin)	Output
1 6	6
2 1	1
3 1	1
4 6	6
5 -1 -2 -3 -4 -5 -6	-1 -2 -3 -4 -5 -6
6 2	2
7 1 -2	1 -2
8 3	3

Advanced String Techniques

Task7: <https://leetcode.com/problems/longest-substring-without-repeating-characters/>

Given a string `s`, find the length of the **longest substring** without duplicate characters.

Example 1:

Input: `s = "abcabcbb"`

Output: 3

Explanation: The answer is "abc", with the length of 3. Note that "bca" and "cab" are also correct answers.

Example 2:

Input: `s = "bbbbbb"`

Output: 1

Explanation: The answer is "b", with the length of 1.

Example 3:

Input: `s = "pwwkew"`

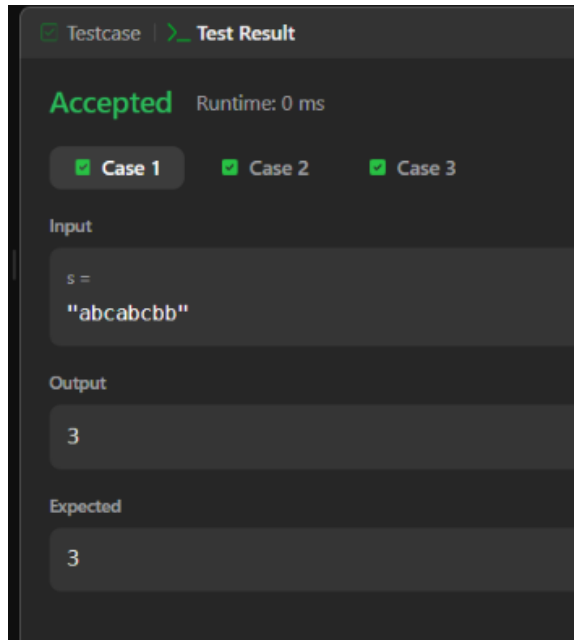
Output: 3

Explanation: The answer is "wke", with the length of 3.

Notice that the answer must be a substring, "pwke" is a subsequence and not a substring.

Program

```
class Solution {  
    public int lengthOfLongestSubstring(String s) {  
        HashSet<Character> set = new HashSet<>();  
        int left = 0;  
        int maxLength = 0;  
        for (int right = 0; right < s.length(); right++) {  
            while (set.contains(s.charAt(right))) {  
                set.remove(s.charAt(left));  
                left++;  
            }  
            set.add(s.charAt(right));  
            maxLength = Math.max(maxLength, right - left + 1);  
        }  
        return maxLength;  
    }  
}
```



Task 8: <https://leetcode.com/problems/find-and-replace-pattern/>

Given a list of strings `words` and a string `pattern`, return a *list of words[i] that match pattern*. You may return the answer in **any order**.

A word matches the pattern if there exists a permutation of letters `p` so that after replacing every letter `x` in the pattern with `p(x)`, we get the desired word.

Recall that a permutation of letters is a bijection from letters to letters: every letter maps to another letter, and no two letters map to the same letter.

Example 1:

Input: `words = ["abc","deq","mee","aqq","dkd","ccc"], pattern = "abb"`

Output: `["mee","aqq"]`

Explanation: "mee" matches the pattern because there is a permutation `{a -> m, b -> e, ...}`.

"ccc" does not match the pattern because `{a -> c, b -> c, ...}` is not a permutation, since a and b map to the same letter.

Example 2:

Input: `words = ["a","b","c"], pattern = "a"`

Output: `["a","b","c"]`

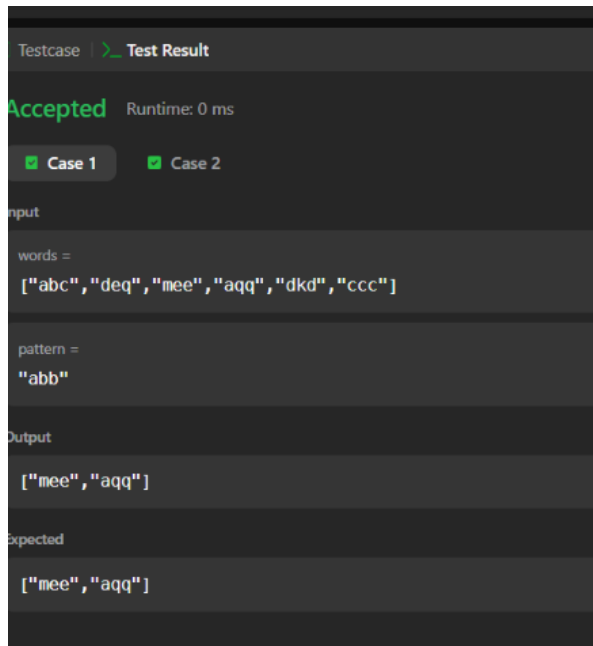
Program

```
class Solution {  
  
    public List<String> findAndReplacePattern(String[] words, String pattern) {  
  
        List<String> result = new ArrayList<>();  
  
        for (String word : words) {  
  
            if (matches(word, pattern)) {  
  
                result.add(word);  
  
            }  
  
        }  
  
        return result;  
  
    }  
  
    public boolean matches(String word, String pattern) {  
  
        HashMap<Character, Character> map1 = new HashMap<>();  
  
        HashMap<Character, Character> map2 = new HashMap<>();  
  
        for (int i = 0; i < word.length(); i++) {  
  
            char w = word.charAt(i);  
  
            char p = pattern.charAt(i);  
  
            // word -> pattern  
  
            if (map1.containsKey(w) && map1.get(w) != p) {  
  
                return false;  
  
            }  
  
            // pattern -> word  
  
            if (map2.containsKey(p) && map2.get(p) != w) {  
  
                return false;  
  
            }  
  
            map1.put(w, p);  
  
        }  
  
    }  
  
}
```

```

        map2.put(p, w);
    }
    return true;
}
}

```



Naive Pattern Matching

Task 9: <https://leetcode.com/problems/string-matching-in-an-array/>

Given an array of string **words**, return all strings in* **words** *that are a substring of another word. You can return the answer in **any order**.

Example 1:

Input: words = ["mass", "as", "hero", "superhero"]

Output: ["as", "hero"]

Explanation: "as" is substring of "mass" and "hero" is substring of "superhero".

["hero", "as"] is also a valid answer.

Example 2:

Input: words = ["leetcode", "et", "code"]

Output: ["et", "code"]

Explanation: "et", "code" are substring of "leetcode".

Example 3:

Input: words = ["blue", "green", "bu"]

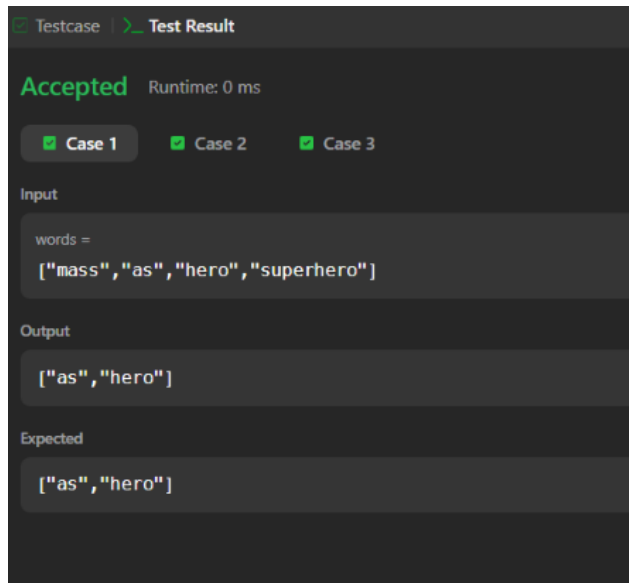
Output: []

Explanation: No string of words is substring of another string.

Program

```
class Solution {  
    public List<String> stringMatching(String[] words) {  
        List<String> result = new ArrayList<>();  
  
        for (int i = 0; i < words.length; i++) {  
            for (int j = 0; j < words.length; j++) {  
  
                if (i != j && words[j].contains(words[i])) {  
                    result.add(words[i]);  
                    break;  
                }  
            }  
        }  
    }  
  
    return result;  
}
```

}



Task10: <https://www.geeksforgeeks.org/dsa/naive-algorithm-for-pattern-searching/>

Given a **text** string of length **n** and a **pattern** of length **m**, find all starting indices where the pattern occurs in the text.

Note: You may assume that $n > m$.

Examples:

Input: text = "geeksforgeeks", pattern = "geeks"

Output: [0, 8]

Explanation: The string "geeks" occurs at index 0 and 8 in text.

Input: text = "aabaacaadaabaaba", pattern = "aaba"

Output: [0, 9, 12]

Program

```
import java.util.ArrayList;
```

```
public class GFG {
```

```
// Function to search for all occurrences of 'pat' in 'txt' using Naive Approach
```

```
static ArrayList<Integer> search(String pat, String txt)
```

```
{
```

```
// Length of the pattern
```

```

int m = pat.length();

// Length of the text
int n = txt.length();

ArrayList<Integer> ans = new ArrayList<>();

// Slide the pattern over text one by one
for (int i = 0; i <= n - m; i++)
{
    int j;

    // Check for pattern match at index i
    for (j = 0; j < m; j++)
    {
        if (txt.charAt(i + j) != pat.charAt(j))
            break;
    }

    // If pattern matches, store index
    if (j == m)
        ans.add(i);
    }

return ans;
}

public static void main(String[] args)
{
    String txt = "aabaacaadaabaaba";
    String pat = "aaba";

    ArrayList<Integer> res = search(pat, txt);

    for (int it : res)
    {
        System.out.print(it + " ");
    }
}

```